

Overview of Virtualization

Edgar Lobaton, Ph.D.

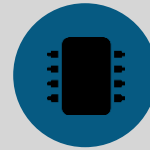
Director of Applied AI, College of Engineering (COE), NC State
Professor, Electrical and Computer (ECE) Engineering, NC State
NAIRR AI Education Fellow, Computing Research Association (CRA)

Why Virtualization Matters



Conflicting Package Versions

Two projects need different versions of the same library on one machine.



CUDA / Driver Mismatches

An installed PyTorch build doesn't match the local CUDA version.



"Works on My Machine"

A script runs fine locally but breaks on a colleague's setup.



Shared Lab Hardware

Multiple users on one GPU workstation or Pi cluster collide.

Three Layers of Isolation

HARDWARE / OS VIRTUALIZATION (VM)

VirtualBox · VMware · Hyper-V · Cloud VMs (AWS EC2, Jetstream2) · WSL2

OS-LEVEL VIRTUALIZATION (CONTAINERS)

Docker · Singularity

LANGUAGE-LEVEL ENVIRONMENTS

venv · conda

The Stack in Practice



Cloud VM

AWS EC2 / Jetstream2 — full OS, GPU-backed hardware



Docker Container

CUDA + PyTorch image — a known-working driver/framework stack



Conda

Project-specific packages — isolated from every other project

These layers aren't competing choices — knowing which one is failing is the real skill.

Where Does Google Colab Fit?

Full Control

Zero Setup



Colab is someone else's VM, pre-configured for you. Zero setup and tied to a Google account participants already have — ideal for workshops and demos, at the cost of less control over the underlying environment.

Which Layer Do You Need?

Just managing Python package versions?



venv / conda

Sharing a full stack with collaborators?



Docker

Deploying to HPC or a shared cluster?



Singularity / Apptainer

Need full hardware isolation or GPU cloud access?



VM (local or cloud)

Want zero setup for a workshop or demo?



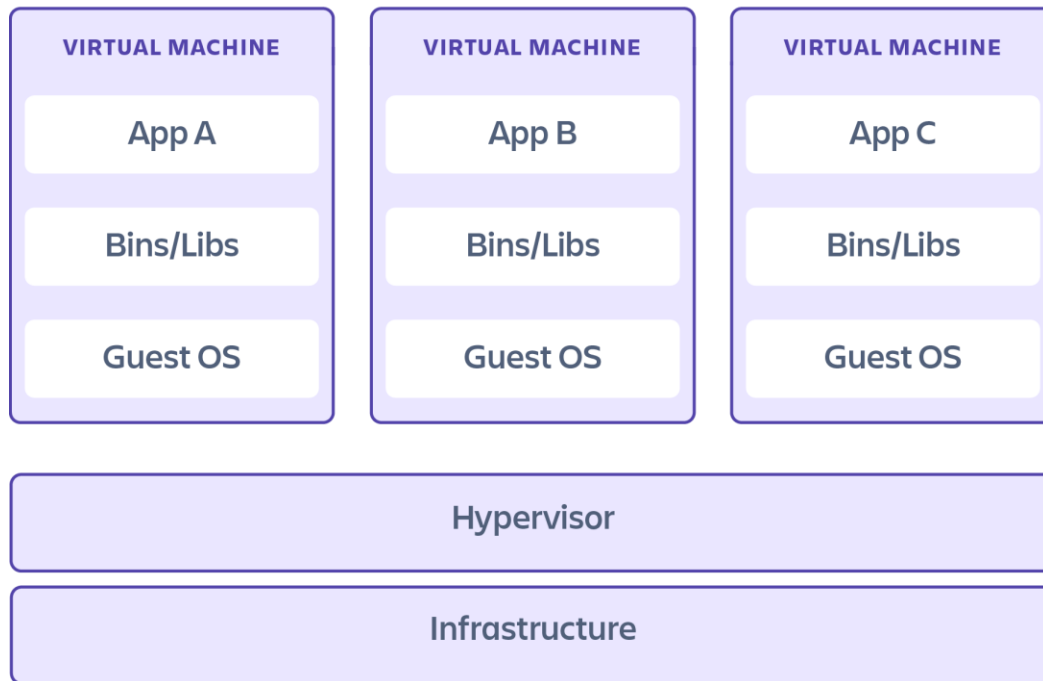
Colab

Hardware and OS Level Virtualization

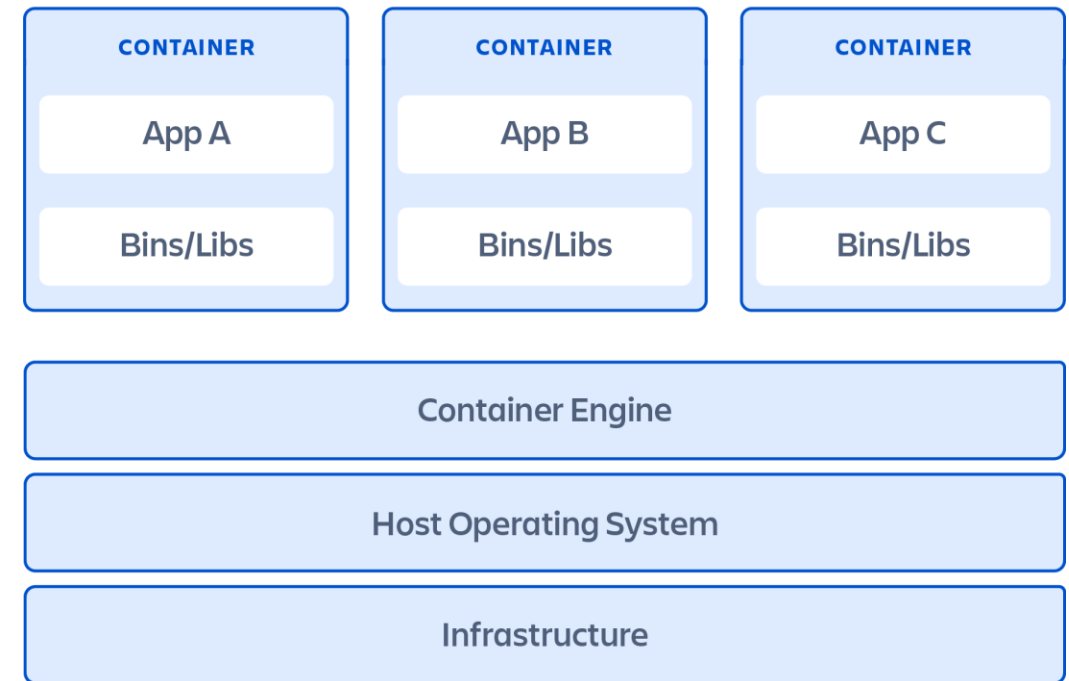
VMs and Containers

Virtual Machines vs. Containers

Virtual machines



Containers



Orchestrates the interactions between the hardware and the VMs

Virtual Machines

VMs virtualize an entire machine down to the hardware layers including CPU, Disk, Memory, and Network devices.

Pros:

- Full isolation security with vulnerabilities with the single system
- Beyond the basic hardware definition, the VM can be treated as a barebone computer.
Good depending on your project's hardware req.

Cons:

- Time consuming to build and regenerate (full stack system)
- More storage, resources and booting time needed

Containers

Containers only virtualize software layers above the OS layer

Pros:

- Lightweight and fast to modify and iterate. Fast to load as well.
- Robust ecosystem with software applications such as databases, messaging systems and scientific software. Useful for microservices.

Cons:

- Vulnerabilities of a container can lead to system vulnerabilities.

Recommendations

- For scientific computing, containers are recommended since they are lightweight and ideal for reproducibility (e.g., packing a predictive AI model, or a visual 3D reconstruction model)
- The **Virtual Computing Lab (VCL)** relies on virtual machines
- The **HPC at NC State** will rely on containers
- For cloud deployments you may need a hybrid approach
- **Kubernetes** is used for scaling deployment of containerized apps

Software-Level Virtualization

venv and conda

Dependency Hell

- Different projects may require different versions of the same library
- This may lead to broken code and difficulty sharing it

PIP – Pip Installs Packages

- It is the standard package manager for Python
- It makes it easy to install, upgrade and uninstall packages from the **Python Package Index** (PyPI), a Python software repository
- It automatically handles dependencies
- Works with *requirements.txt* allowing to replicate environments

Environments to the Rescue

- Different projects may have different libraries and dependencies... **Environments** are the solution.
- You can use the *requirements.txt* file to export the packages used and reproduce in a new environment.
- This helps keep your Python installation clean

Always use environments for every new project

Environments to the Rescue

VENV – Virtual Environments

- Built-in tool for creating environments.
- It creates a dedicated directory containing a copy of the Python interpreter and a pip installer.
- Any packages installed within this environment will be installed in the dedicated directory, leaving the global Python clean.
- Create: `python -m venv <directory_name>`
- Activate: `source <directory_name>/bin/activate`

Environments to the Rescue

CONDA – Package and Environment Manager

- It is like pip but it can handle Python, R, Java, C++, etc. environments
- It handles more complex dependencies using Conda-Forge repo
- Requires a separate installation: Anaconda or Miniconda
- Platforms such as **Anaconda** are great for scientific computing including environments with the basic data science libraries.
- Create: `conda create -n <env_name> python=<version>`
- Activate: `conda activate <env_name>`

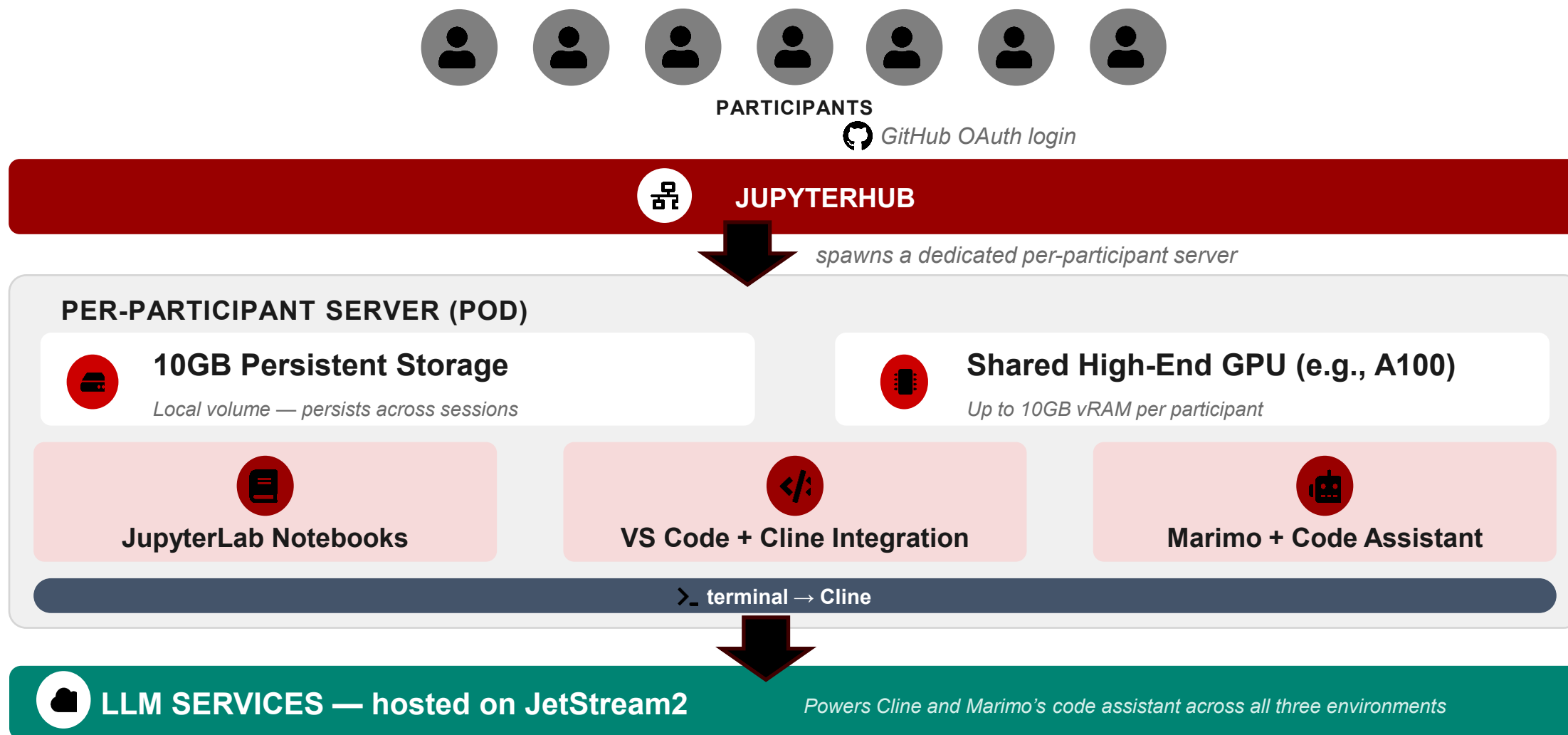
Recommendations

- **Always use environments for every new project**
- Use conda first for package installation
- Use pip as a back up within a conda environment
- Conda is smart enough to track packages installed with pip as well
- Make sure to create *requirements.txt* files within repos for reproducibility.
- Conda has an equivalent *yml* file used to reproduce environments.

JetStream2 Platform

Application of the Three-Levels of Virtualization

JetStream2 Workspace Environment



Marimo vs Jupyter: Two Notebook Models

	Jupyter	Marimo
Execution model	Linear — cells can be run in any order	Reactive — dependent cells auto-update, like a spreadsheet
File format	.ipynb (JSON) — messy git diffs	.py (plain Python) — git-friendly, IDE-native
Reproducibility	Hidden state is a well-known failure mode	No hidden state — a dependency graph is enforced
Deployment	Needs extra tooling to become an app	Runs as a web app or script — no rewrite needed
Working with AI agents	JSON format complicates automated edits	Plaintext — agents like Claude Code can read and edit it directly
Best for	Widest ecosystem — the standard since 2014	Reactive dashboards, reproducible pipelines, agent-friendly workflows

Marimo doesn't match Jupyter's ecosystem maturity yet — but its reactive, git-friendly design solves the reproducibility problems most notebooks eventually run into.

Let's Try it Out